Theorie

Vom Forward-Pass zur Backpropagation

Inhaltsverzeichnis

Bezeichnungen 1
Forward-Pass 1
Kostenfunktion 2
Backpropagation 2
- Ableitung der Sigmoid-Funktion 2
- Output-Layer 2
- Hidden-Layer 3
- Gradienten 3
- Update 3

Prüfung der Herleitung 3
Lokale Minima 4

Die Angabe leitet alles für ein Netz mit genau einem Hidden-Layer her. Hier steht dieselbe Rechnung allgemein für beliebig viele Layer und Neuronen — genau so ist sie auch im Code umgesetzt.

Bezeichnungen

Symbol	Bedeutung
$A^{[0]} = X$	Input des Netzes, z. B. $[0, 1]$ beim XOR
$W^{[l]}$, $b^{[l]}$	Gewichtsmatrix und Bias-Vektor von Layer l; Zeile j von $W^{[l]}$ gehört zu Neuron j
$Z^{[l]}$	gewichtete Summe von Layer l, vor der Aktivierung
$A^{[l]}$	Ausgabe (Aktivierung) von Layer l
L	Nummer des letzten Layers; $A^{[L]} = \hat{Y}$ ist die Vorhersage
Y	gewünschte Ausgabe (Zielwert)
α	Lernrate, hier 0,1

Der Input ist **kein eigener Layer**: Er hat keine Gewichte und rechnet nichts. „Input layer: 2 neurons„ in der Angabe heißt also nur, dass das Netz zwei Eingabewerte bekommt.

Forward-Pass

Jeder Layer rechnet zuerst eine gewichtete Summe und wendet dann die Aktivierungsfunktion an:

$$Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]} \tag{1}$$

$$A^{[l]} = \sigma\big(Z^{[l]}\big), \qquad \sigma(z) = \frac{1}{1+e^{-z}} \tag{2}$$

Für ein einzelnes Neuron j ist das $z_j = \sum_i w_{ji}\, a_i + b_j$ — ein Skalarprodukt plus Bias. Die Sigmoid-Funktion drückt jeden Wert in das Intervall $(0, 1)$; ohne sie wäre das ganze Netz nur eine einzige lineare Funktion und könnte XOR nicht lernen.

```
flowchart LR
    X["X = A⁰"] --> Z1["Z¹ = W¹X + b¹"] --> A1["A¹ = σ(Z¹)"] --> Z2["Z² = W²A¹ + b²"] --> A2["A² = Ŷ"] --> J["Kosten J"]
```

Kostenfunktion

Wie weit die Vorhersage vom Ziel entfernt ist, misst die Kostenfunktion aus der Angabe (Summe der quadratischen Fehler, halbiert):

$$J = \frac{1}{2} \sum_k \left(\hat{y}_k - y_k\right)^2 \tag{3}$$

Der Faktor $\frac{1}{2}$ ist Bequemlichkeit: Beim Ableiten fällt er gegen die 2 aus dem Quadrat weg, es bleibt $\frac{\partial J}{\partial \hat{y}_k} = \hat{y}_k - y_k$.

Trainiert wird mit **stochastischem Gradientenabstieg (SGD)**: nach jedem einzelnen Beispiel ein Update. Eine **Epoche** ist ein Durchlauf über alle Beispiele — beim XOR also 4 Updates, beim 7-Segment-Decoder 10.

Backpropagation

Gesucht ist für jedes Gewicht, wie stark sich J ändert, wenn man das Gewicht ein wenig ändert: $\frac{\partial J}{\partial W^{[l]}}$ und $\frac{\partial J}{\partial b^{[l]}}$. Die Kettenregel zerlegt das in Faktoren entlang des Rechenwegs. Zentral ist dabei eine Hilfsgröße pro Neuron, das **Delta**:

$$\delta^{[l]} = \frac{\partial J}{\partial Z^{[l]}}$$

Ableitung der Sigmoid-Funktion

$$\sigma'(z) = \sigma(z)\big(\,1 - \sigma(z)\big) = a\,(1-a) \tag{4}$$

Die Ableitung lässt sich also direkt aus der **Ausgabe** a berechnen, die das Neuron ohnehin speichert. Wichtig: a ist schon $\sigma(z)$ — die Sigmoid-Funktion darf hier nicht ein zweites Mal angewendet werden.

Output-Layer

Kettenregel aus Kosten und Aktivierung, elementweise ($\odot$):

$$\delta^{[L]} = \underbrace{\left(A^{[L]} - Y\right)}_{\partial J / \partial A^{[L]}} \odot \underbrace{A^{[L]}\left(1 - A^{[L]}\right)}_{\partial A^{[L]} / \partial Z^{[L]}} \tag{5}$$

Hidden-Layer

Ein Hidden-Neuron beeinflusst die Kosten über **alle** Neuronen des nächsten Layers. Deren Deltas fließen gewichtet zurück:

$$\delta^{[l]} = \left(W^{[l+1]\top}\delta^{[l+1]}\right) \odot A^{[l]}\left(1 - A^{[l]}\right) \tag{6}$$

Der Ausdruck in der Klammer ist der „Fehler„ $\frac{\partial J}{\partial A^{[l]}}$, den ein Layer an seinen Vorgänger zurückgibt — im Code der Rückgabewert von `Layer.backward`.

Gradienten

Wegen $Z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]}$ gilt $\frac{\partial z_j}{\partial w_{ji}} = a_i^{[l-1]}$ und $\frac{\partial z_j}{\partial b_j} = 1$:

$$\frac{\partial J}{\partial w_{ji}^{[l]}} = \delta_j^{[l]}\, a_i^{[l-1]} \qquad \frac{\partial J}{\partial b_j^{[l]}} = \delta_j^{[l]} \tag{7}$$

Für das Netz der Angabe mit einem Hidden-Layer ($L = 2$) ergibt das genau die Formeln aus dem PDF, zum Beispiel

$$\frac{\partial J}{\partial W^{[1]}} = \left(A^{[2]} - Y\right) \cdot A^{[2]}\left(1 - A^{[2]}\right) \cdot W^{[2]} \cdot A^{[1]}\left(1 - A^{[1]}\right) \cdot X$$

Update

Jedes Gewicht macht einen kleinen Schritt gegen den Gradienten:

$$W_{\text{neu}}^{[l]} = W_{\text{alt}}^{[l]} - \alpha\,\frac{\partial J}{\partial W^{[l]}} \qquad b_{\text{neu}}^{[l]} = b_{\text{alt}}^{[l]} - \alpha\,\frac{\partial J}{\partial b^{[l]}} \tag{8}$$

Der Fehler für den vorherigen Layer (Gleichung 6) muss mit den Gewichten **aus dem Forward-Pass** berechnet werden, also **bevor** die Gewichte dieses Layers aktualisiert werden. Andersherum mischt man alte Aktivierungen mit neuen Gewichten, und der Gradient stimmt nicht mehr.

Prüfung der Herleitung

Eine falsche Ableitung fällt beim Training oft gar nicht auf — das Netz lernt dann eben schlechter. Deshalb vergleicht ein Test die Backpropagation mit einer **numerischen Ableitung**: Jedes Gewicht wird um $\varepsilon = 10^{-6}$ nach oben und unten verschoben und die Kosten werden neu berechnet.

$$\frac{\partial J}{\partial w} \approx \frac{J(w + \varepsilon) - J(w - \varepsilon)}{2\varepsilon}$$

Für drei verschiedene Netzgrößen (2-2-1, 4-16-7 und 3-5-4-2 mit zwei Hidden-Layern) stimmen beide Werte auf mindestens 10^{-8} überein.

Eine zweite, unabhängige Prüfung liefert der Vergleich mit TensorFlow: TensorFlow bildet die Ableitungen automatisch und folgt mit denselben Startgewichten über alle 20 000 Epochen demselben Verlauf wie die eigene Implementierung.

Lokale Minima

Gradientenabstieg findet ein Minimum in der Nähe des Startpunkts, nicht unbedingt das beste. Gerade das kleine XOR-Netz mit nur zwei Hidden-Neuronen hat Bereiche, in denen der Gradient fast null ist, obwohl das Problem nicht gelöst ist: Das Netz gibt dann für zwei der vier Inputs ungefähr 0,5 aus und kommt dort nicht mehr weg. Ob das passiert, hängt von den zufälligen Startgewichten ab — wie oft, zeigen die Ergebnisse.